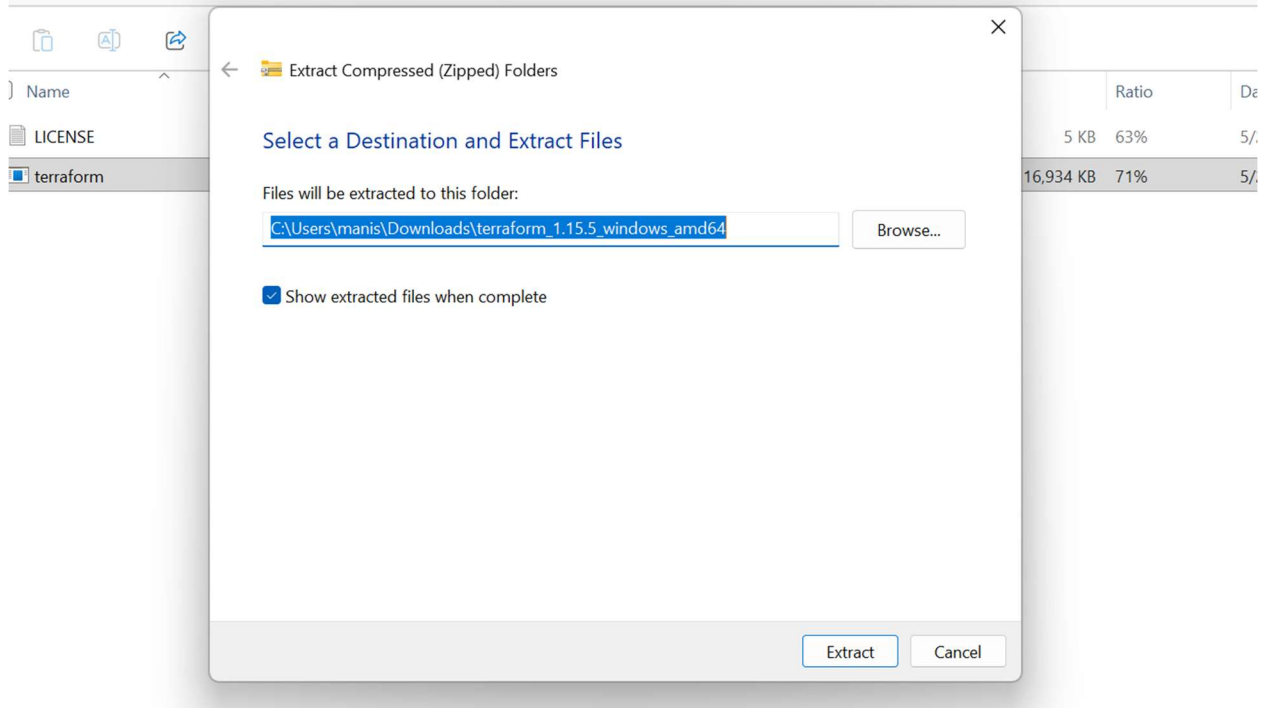
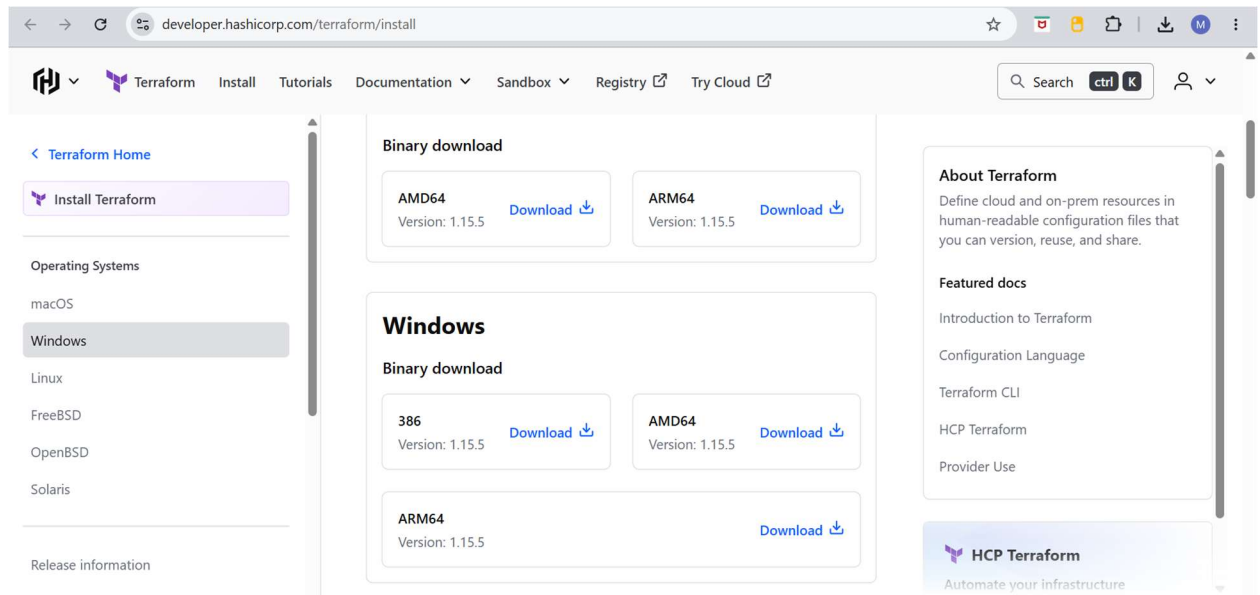


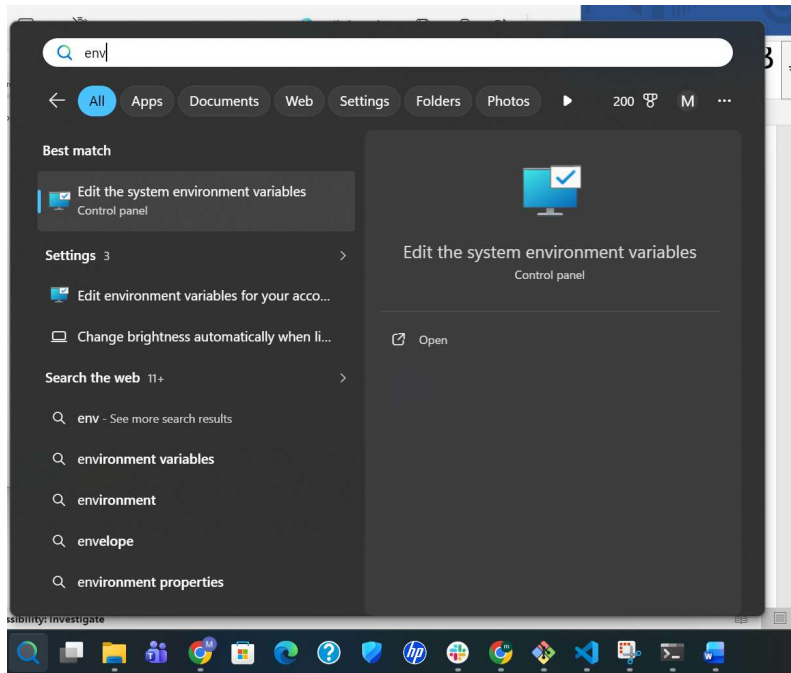
TERRAFORM-01

1). Install Terraform on your PC

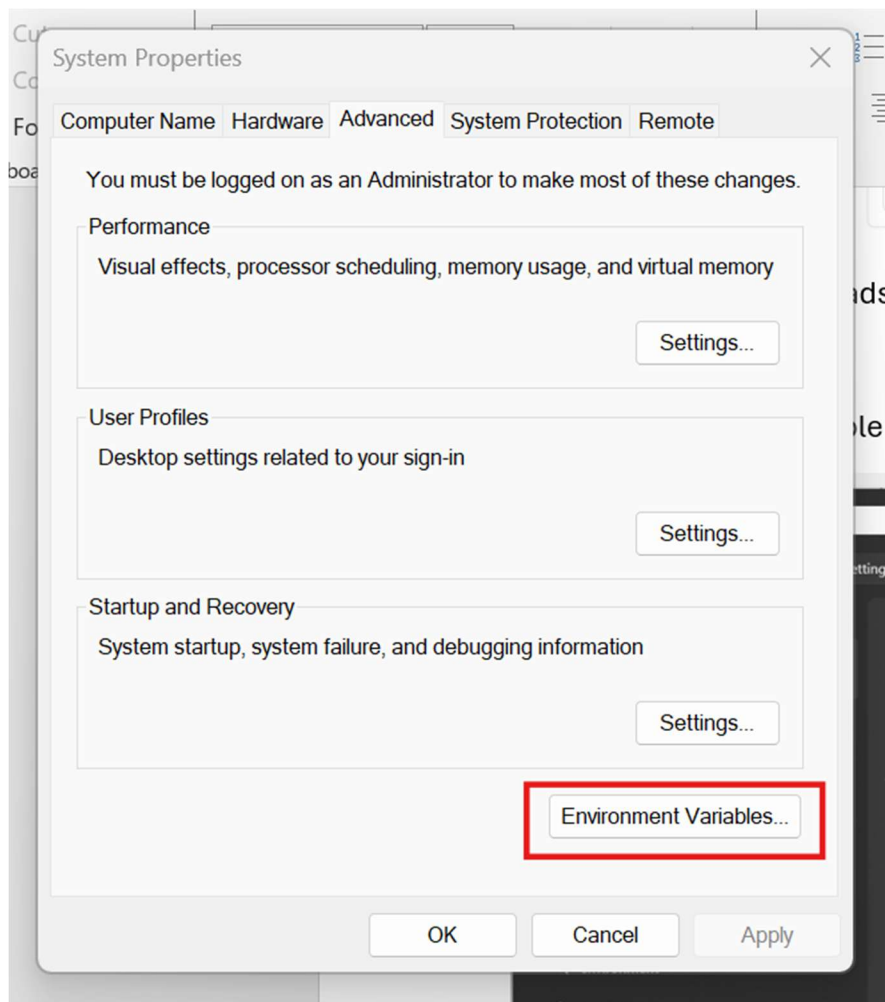
→ First let's download terraform



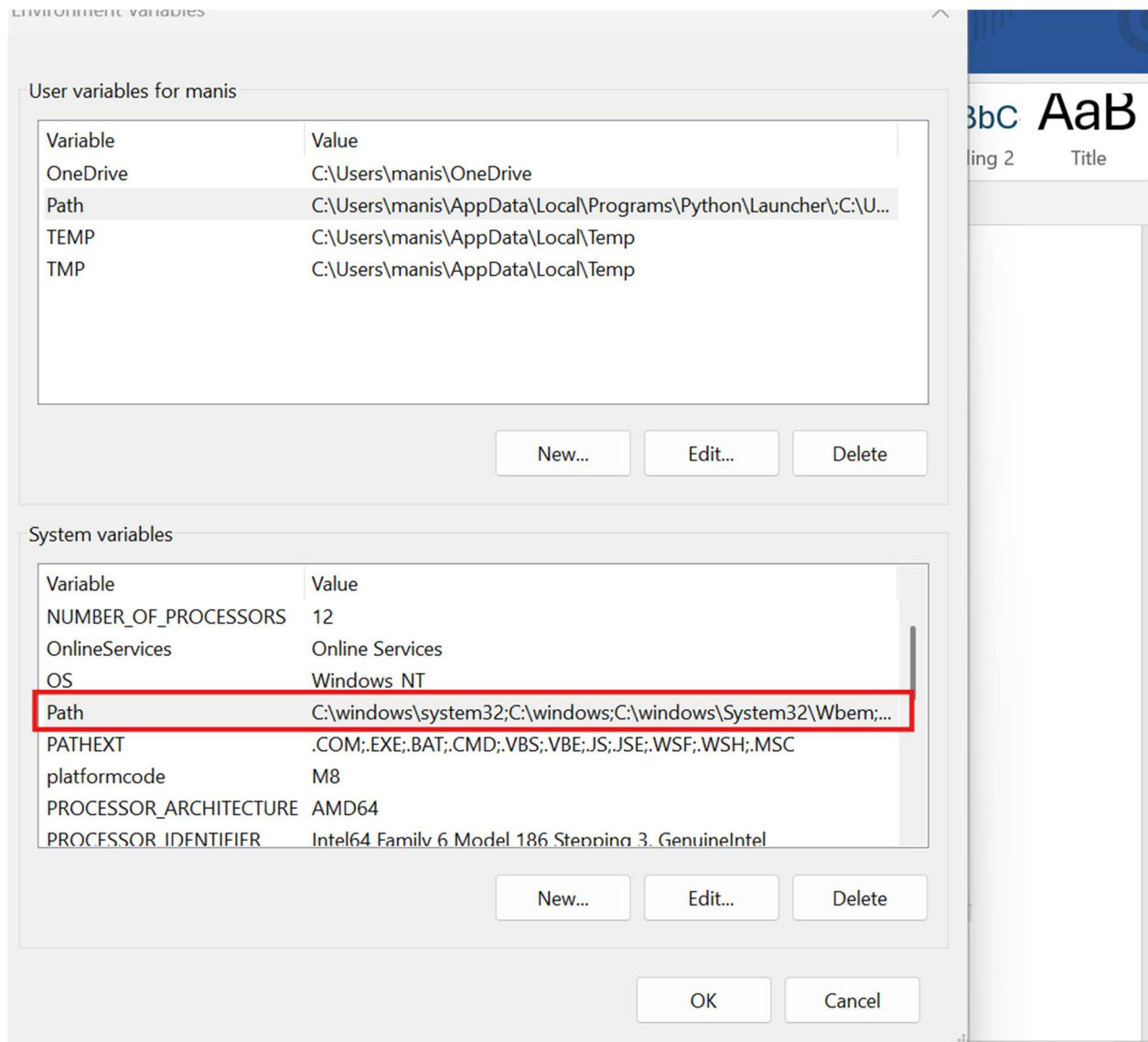
→Open environment variable



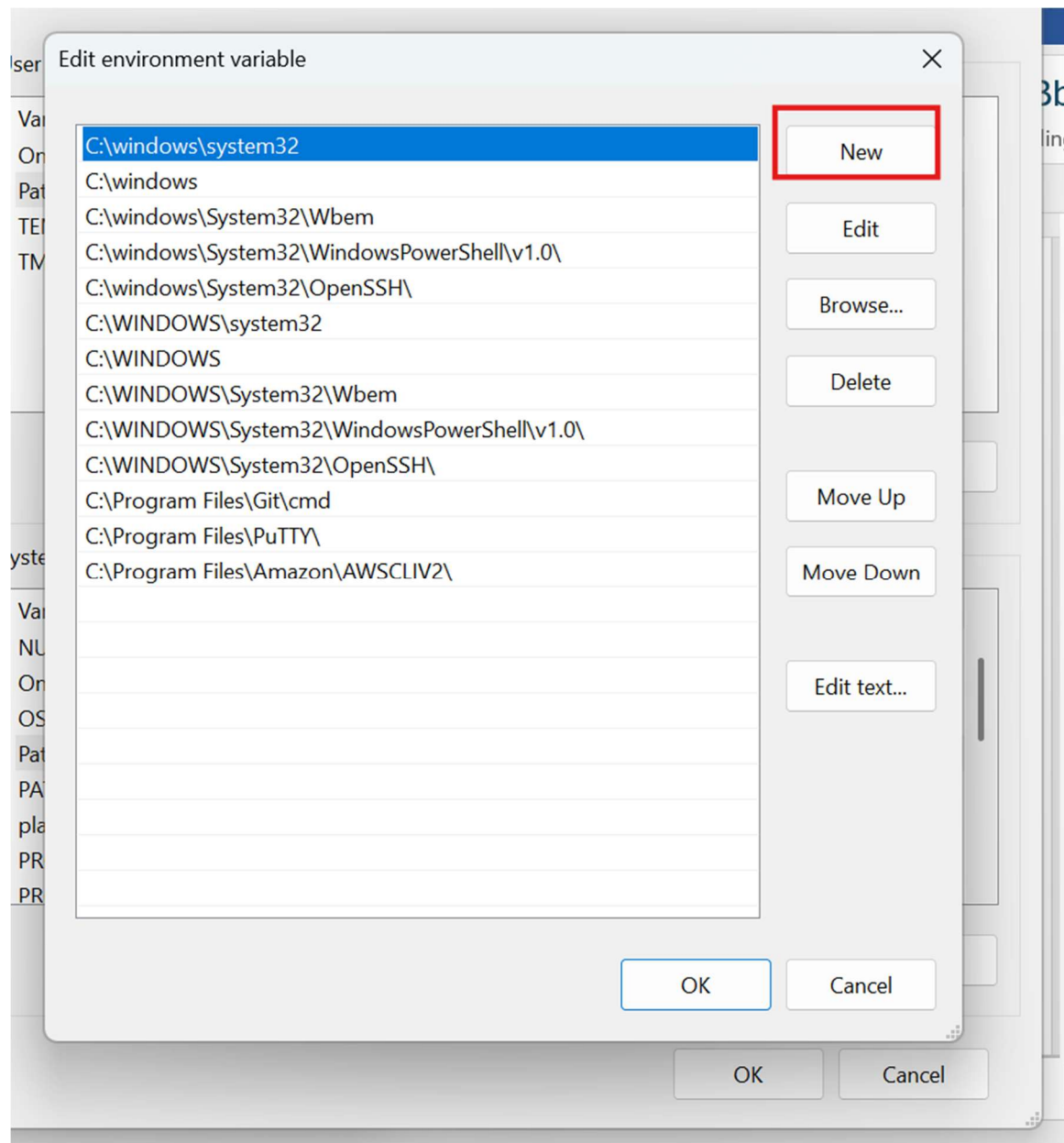
→Choose environment variable



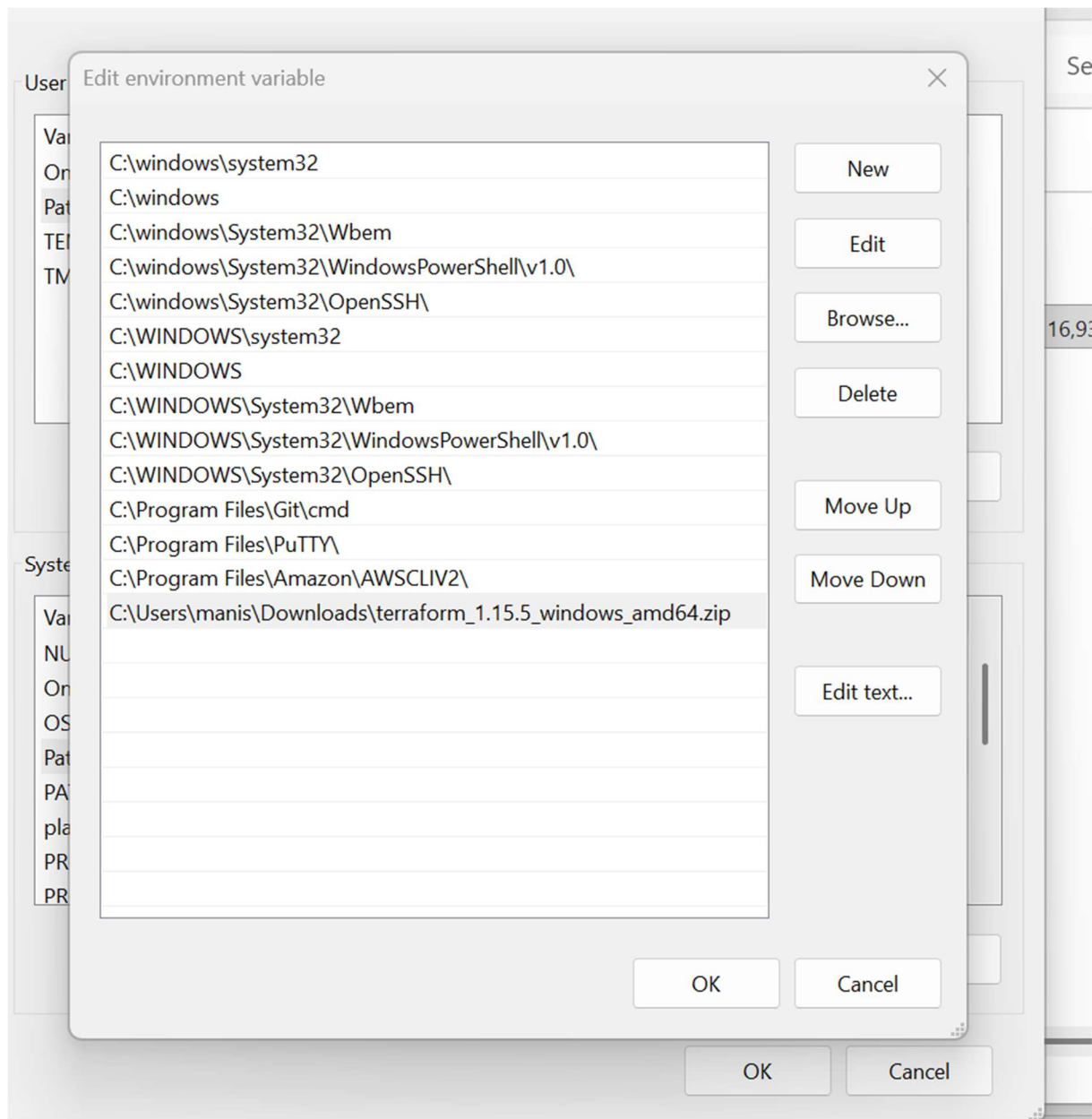
Click on edit



Click on new



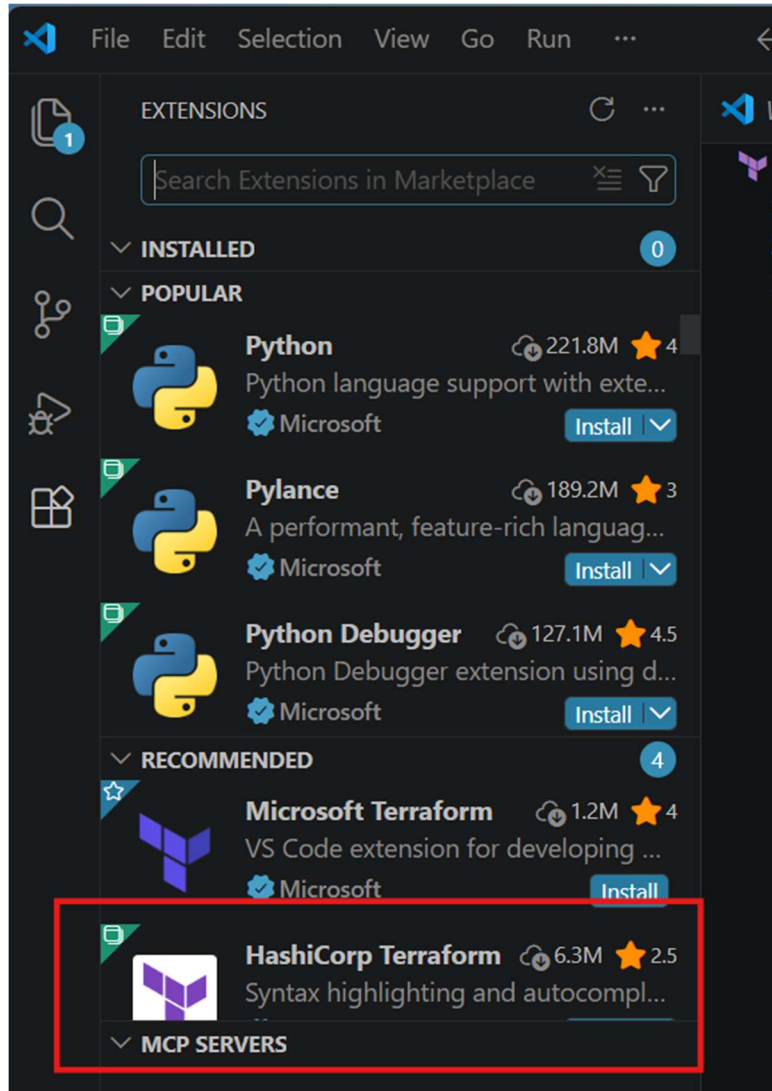
Add the path and click on ok



→ Now let's verify the version

```
MINGW64:/c/Users/manis/Downloads
manis@MANISHAREDDY MINGW64 ~/Downloads
$ terraform -v
Terraform v1.15.5
on windows_amd64
manis@MANISHAREDDY MINGW64 ~/Downloads
$ |
```

2). Execute all the templates shown in video.





HashiCorp Terraform

HashiCorp  hashicorp.com | 6,390,131 | (207)

Syntax highlighting and autocompletion for Terraform

Do you trust the publisher "HashiCorp"?

The extension HashiCorp Terraform is published by HashiCorp. This is the first extension you're installing from this publisher.

 HashiCorp has verified ownership of www.hashicorp.com.

Visual Studio Code has no control over the behavior of third-party extensions, including how they manage your personal data. Proceed only if you trust the publisher.

[Trust Publisher & Install](#) [Learn More](#) [Cancel](#)

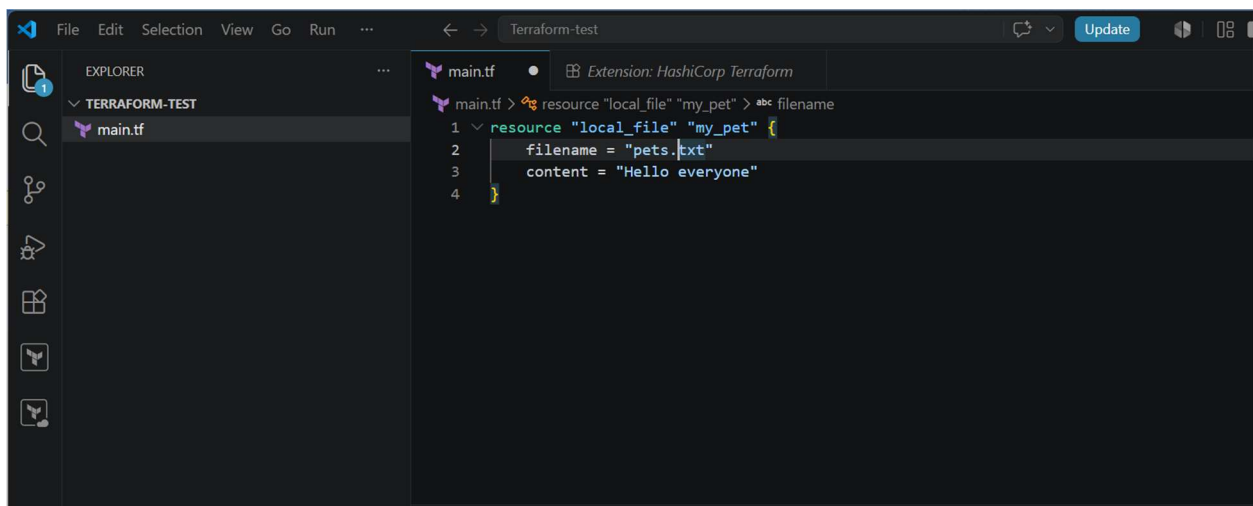
Quick Start

Get started writing Terraform configurations with VS Code in three steps:

Categories

- Programming Languages
- Linters
- Formatters

Resources



```
main.tf
resource "local_file" "my_pet" {
  filename = "pets.txt"
  content = "Hello everyone"
}
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + v [ ] [ ] ... [ ] [ ]
PS C:\Users\manis\Desktop\Terraform-test> terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/local...
- Installing hashicorp/local v2.9.0...
- Installed hashicorp/local v2.9.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
```

Terraform plan

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + v [ ] [ ] ... [ ] [ ]
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\manis\Desktop\Terraform-test> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# local_file.my_pet will be created
+ resource "local_file" "my_pet" {
  + content           = "Hello everyone"
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5       = (known after apply)
  + content_sha1      = (known after apply)
  + content_sha256    = (known after apply)
  + content_sha512    = (known after apply)
  + directory_permission = "0777"
  + file_permission   = "0777"
```

terraform apply

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + v [ ] [ ] ... [ ] [ ] X

PS C:\Users\manis\Desktop\Terraform-test> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# local_file.my_pet will be created
+ resource "local_file" "my_pet" {
+   content           = "Hello everyone"
+   content_base64sha256 = (known after apply)
+   content_base64sha512 = (known after apply)
+   content_md5       = (known after apply)
+   content_sha1      = (known after apply)
+   content_sha256    = (known after apply)
+   content_sha512    = (known after apply)
+   directory_permission = "0777"
+   file_permission   = "0777"
+   filename          = "pets.txt"
+   id                = (known after apply)
}
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + v [ ] [ ] ... [ ] [ ] X

+ content_sha256      = (known after apply)
+ content_sha512     = (known after apply)
+ directory_permission = "0777"
+ file_permission    = "0777"
+ filename           = "pets.txt"
+ id                 = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

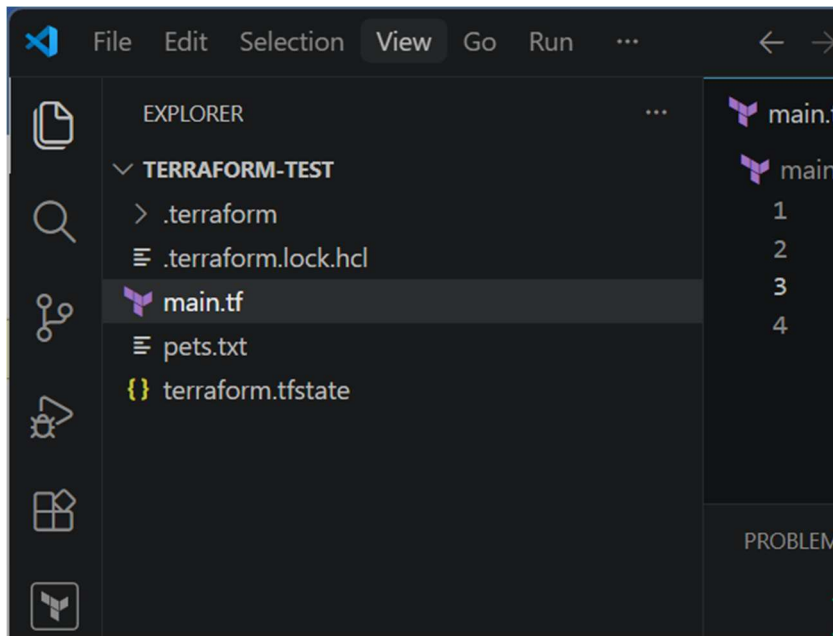
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=64aa4395c9ec959f8616c5bb40ec9b0587b9f80b]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\manis\Desktop\Terraform-test> [ ]
```

File has been created successfully



So I am changing the content from hello everyone to I love pets

```
main.tf x terraform.tfstate
main.tf resource "local_file" "my_pet" {
1  resource "local_file" "my_pet" {
2      filename = "pets.txt"
3      content = "I love pets"
4  }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\manis\Desktop\Terraform-test> terraform plan
local_file.my_pet: Refreshing state... [id=64aa4395c9ec959f8616c5bb40ec9b0587b9f80b]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

# local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
  ~ content          = "Hello everyone" -> "I love pets" # forces replacement
  ~ content_base64sha256 = "NB2URXebGfite/qTzyKswgWK8TQH6vwBBtZ10f5bsrk=" -> (known after apply)
  ~ content_base64sha512 = "eazPm5h3hA/HQ3Uln7k7TMEgI7k9Nwq3EaBCTVxZchAn19X0X4Idt78TvVu398v0Db/7UzBd2Dz3j5awk7c4Cg=="
  -> (known after apply)
  ~ content_md5       = "83cb6f705911f7dac48af0a87abc8b3b" -> (known after apply)
  ~ content_sha1       = "64aa4395c9ec959f8616c5bb40ec9b0587b9f80b" -> (known after apply)
  ~ content_sha256     = "341d9445779b19f8ad7bfa93cf22acc2058af13407eafc0106d675d1fe5bb2b9" -> (known after apply)
  ~ content_sha512     = "79accf9b9877840fc74375259fb93b4cc12023b93d370ab711a0424d5c5972102797d5f45f821db7bf13bd5bb7
f7cbf40dbff53305dd83cf78f96b093b7380a" -> (known after apply)
  ~ id                = "64aa4395c9ec959f8616c5bb40ec9b0587b9f80b" -> (known after apply)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
~ content_sha512      = "79accf9b9877840fc74375259fb93b4cc12023b93d370ab711a0424d5c5972102797d5f45f821db7bf13bd5
f7cbf40dbffb53305dd83cf78f96b093b7380a" -> (known after apply)
~ id                  = "64aa4395c9ec959f8616c5bb40ec9b0587b9f80b" -> (known after apply)
# (3 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

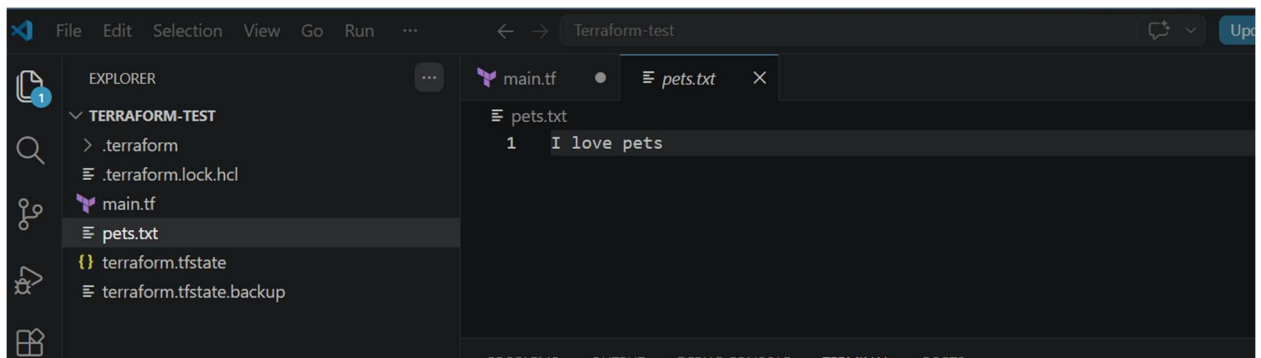
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

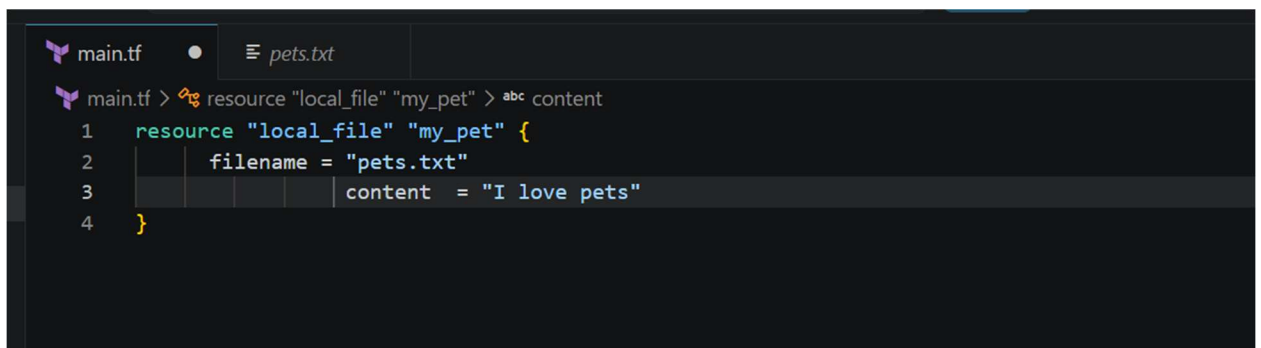
local_file.my_pet: Destroying... [id=64aa4395c9ec959f8616c5bb40ec9b0587b9f80b]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=4246687863ee17bf4daf5fc376ab01222e989aca]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\Users\manis\Desktop\Terraform-test>
```

→Successfully changed the content



→I want the code to be aligned properly

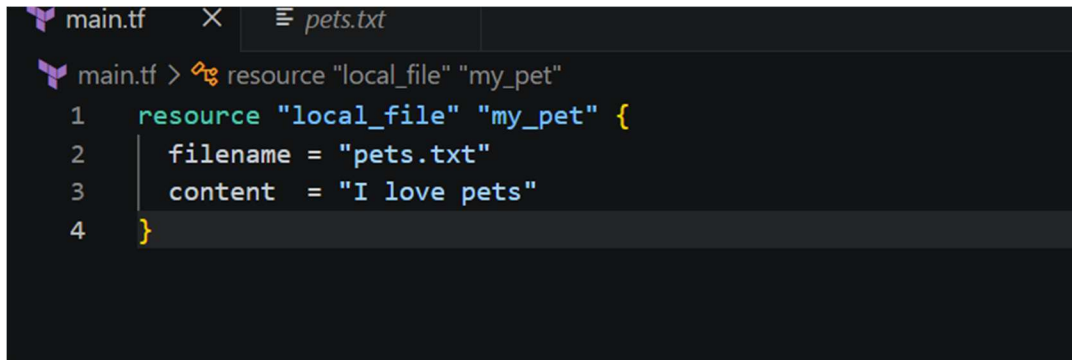


→For formatting we use below code

terraform fmt

```
PS C:\Users\manis\Desktop\Terraform-test> terraform fmt
main.tf
PS C:\Users\manis\Desktop\Terraform-test> █
```

→Aligned successfully



```
main.tf  X  pets.txt
main.tf > resource "local_file" "my_pet"
1  resource "local_file" "my_pet" {
2      filename = "pets.txt"
3      content  = "I love pets"
4  }
```

3. Note down below points, Terraform Init Terraform Plan Terraform Apply Terraform Provider

Terraform init

Purpose:

- Initializes the Terraform working directory.
- Downloads required provider plugins.
- Configures the backend for state storage.
- Downloads referenced modules.

When to Use:

- Before running any Terraform commands in a new project.
- After adding or changing providers/modules.

Terraform plan

Purpose:

- Creates an execution plan.
- Shows what resources will be created, modified, or destroyed.
- Does not make any actual changes.

When to Use:

- Before applying changes to review the impact.

Terraform apply

Purpose:

- Executes the changes defined in the Terraform configuration.
- Creates, updates, or deletes infrastructure resources.
- Updates the Terraform state file.

When to Use:

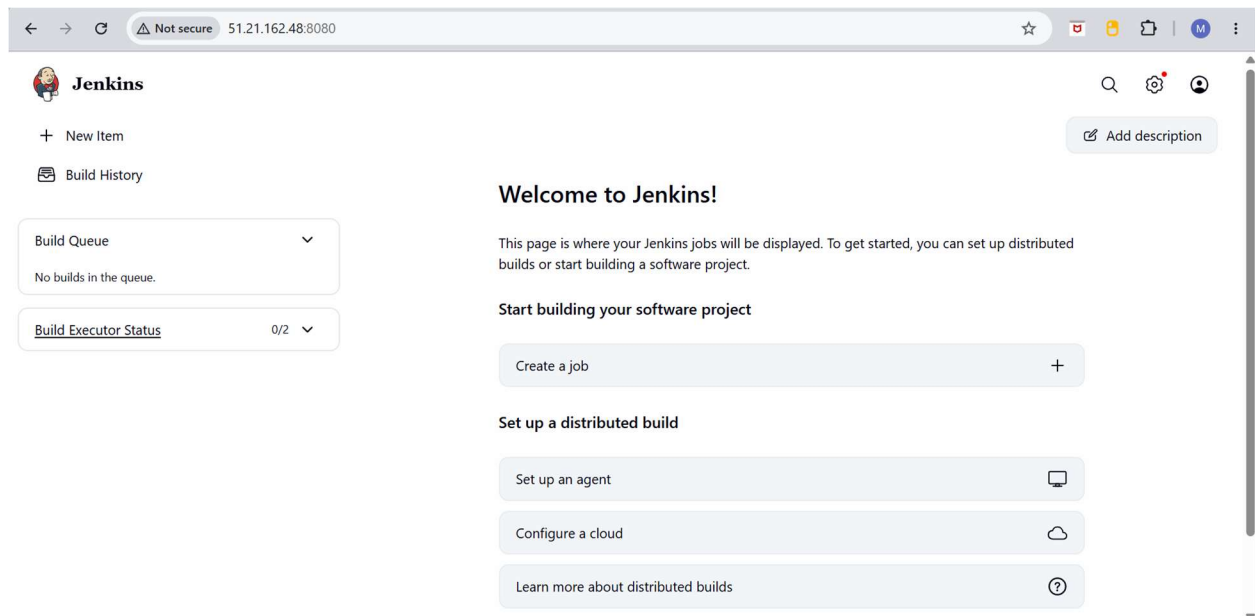
- After reviewing the execution plan.

Terraform provider : It is a complete package of api calls to communicate with our resources

→ Acts as the bridge between Terraform and the target infrastructure platform.

4.Integrate a sample Terraform template in Jenkins.

Lets login into jenkins



Lets create a main.tf

```

Terraform v1.15.5
on linux_amd64
root@ip-172-31-40-52:/home/ubuntu# ls
root@ip-172-31-40-52:/home/ubuntu# cd /var/lib/jenkins
root@ip-172-31-40-52:/var/lib/jenkins# ls
caches
config.xml
hudson.model.UpdateCenter.xml
hudson.plugins.git.GitTool.xml
identity.key.enc
jenkins.install.InstallUtil.lastExecVersion
jenkins.install.UpgradeWizard.state
jenkins.model.JenkinsLocationConfiguration.xml
jenkins.telemetry.Correlator.xml
jobs
logs
nodeMonitors.xml
org.jenkinsci.plugins.workflow.flow.FlowExecutionList.xml
plugins
queue.xml
secret.key
secret.key.not-so-secret
secrets
updates
userContent
users
workspace
root@ip-172-31-40-52:/var/lib/jenkins# cd workspace/
root@ip-172-31-40-52:/var/lib/jenkins/workspace# ls
jenkins  jenkins@tmp
root@ip-172-31-40-52:/var/lib/jenkins/workspace# cd jenkins
root@ip-172-31-40-52:/var/lib/jenkins/workspace/jenkins# ls
root@ip-172-31-40-52:/var/lib/jenkins/workspace/jenkins# vi main.tf

```

Lets write a script in main.tf

```

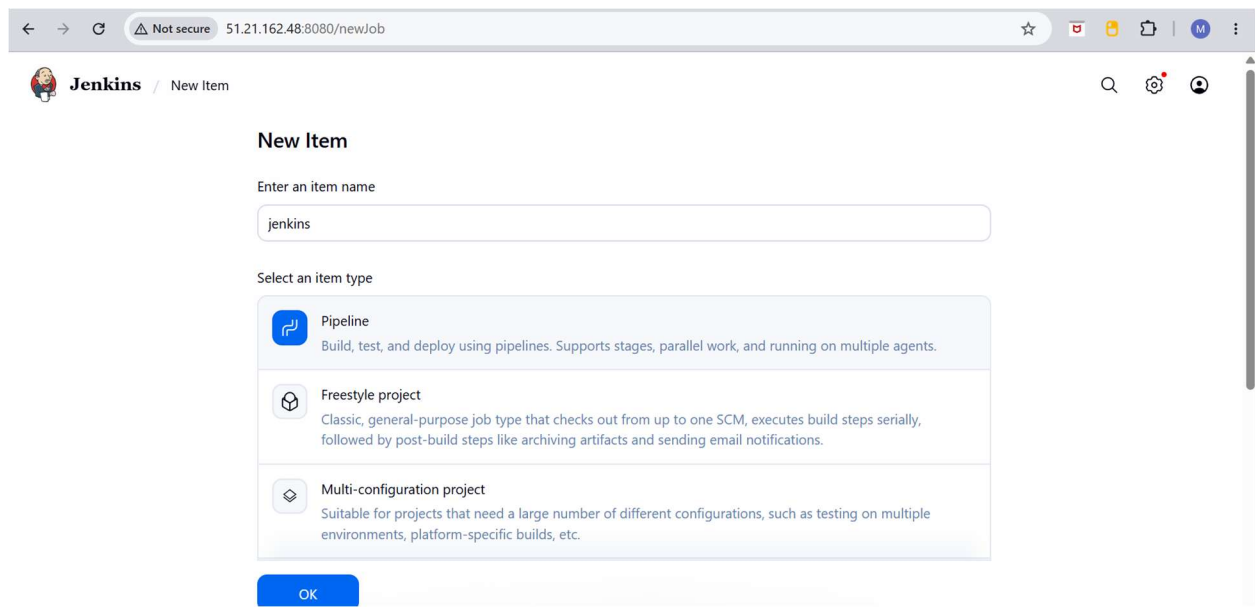
root@ip-172-31-40-52: /var/lib/jenkins/workspace/jenkins
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
    }
  }
}

resource "local_file" "demo" {
  filename = "hello.txt"
  content  = "Hello from Terraform and Jenkins!"
}
~
~
~
~
~

```

This Terraform script creates a file named **hello.txt** and writes **"Hello from Terraform and Jenkins!"** into it. Jenkins runs terraform init, plan, and apply to automatically create the file, demonstrating basic Terraform automation.

Now lets create a pipeline



→Write a pipeline script

```
pipeline {  
  agent any
```

```
  stages {  
    stage('Terraform Init') {  
      steps {  
        sh 'terraform init'  
      }  
    }  
  }
```

```
  stage('Terraform Plan') {  
    steps {  
      sh 'terraform plan'  
    }  
  }  
}
```

```
stage('Terraform Apply'){  
  steps{  
    sh 'terraform apply -auto-approve'  
  }  
}  
}  
}
```

Script ?

```
1 pipeline {  
2   agent any  
3  
4   stages {  
5     stage('Terraform Init') {  
6       steps {  
7         sh 'terraform init'  
8       }  
9     }  
10  
11    stage('Terraform Plan') {  
12      steps {  
13        sh 'terraform plan'  
14      }  
15    }  
}
```

Script ?

```
9      }
10
11      stage('Terraform Plan') {
12          steps {
13              sh 'terraform plan'
14          }
15      }
16
17      stage('Terraform Apply') {
18          steps {
19              sh 'terraform apply -auto-approve'
20          }
21      }
22  }
23 }
```



Use Groovy Sandbox ?

→ I integrated Terraform with Jenkins by creating a Jenkins pipeline that executes terraform init, terraform plan, and terraform apply stages automatically whenever the job runs.

→ Now let us build job

The screenshot shows the Jenkins web interface for job #2. The browser address bar indicates the URL is 51.21.162.48:8080/job/jenkins/2/. The Jenkins logo and job name 'jenkins' are at the top. A sidebar on the left contains links to 'Status', 'Changes', 'Console Output', 'Edit Build Information', 'Delete build #2', 'Timings', 'Pipeline Overview', 'Restart from Stage', 'Replay', and 'Pipeline Steps'. The main content area shows a green checkmark icon and the text '#2 (09-Jun-2026, 1:03:02 pm)'. Below this, it says 'Started by user admin' and 'This run spent: 11 ms waiting; 1.7 sec build duration; 1.7 sec total from scheduled to completion.' There is also a 'No changes' message. On the right, there are buttons for 'Add description' and 'Keep this build forever', along with a search icon, a settings icon, and a user icon.

Job ran succesfully

Here we can able to see the created .txt file

```
root@ip-172-31-40-52:/var/lib/jenkins/workspace/jenkins# ls
hello.txt  main.tf  terraform.tfstate
root@ip-172-31-40-52:/var/lib/jenkins/workspace/jenkins# cat hello.txt
Hello from Terraform and Jenkins!root@ip-172-31-40-52:/var/lib/jenkins/workspace
root@ip-172-31-40-52:/var/lib/jenkins/workspace/jenkins#
```